

Module 5 Collaborative Delivery and Operation

Overview and study route

Software Engineering in the Era of AI | Master's year one | Electrical and Information Engineering | Weeks 11 and 12 | 2026 to 2027

A program that works once on its author's laptop is not yet an operational service. Another person needs to reproduce the environment, recognise the deployed version, detect a failure and recover a known state. AI can accelerate changes, which makes these controls more valuable because a team may otherwise generate changes faster than it can inspect them.

Week 11 connects collaborative work to a repeatable quality gate. Week 12 develops deployment, observability and recovery. The required practical route is local and reproducible; containers and managed cloud environments are optional ways to implement the same responsibilities.

Learning outcomes

- Plan a small increment with acceptance criteria and a shared definition of completion.
- Distinguish continuous integration, delivery and deployment.
- Build and explain a repeatable quality gate and dependency record.
- Describe the purpose and limits of build provenance and a software bill of materials.
- Design useful logs, metrics and traces without exposing unnecessary data.
- Define a service objective and calculate a simple error budget.
- Rehearse rollback and write an evidence-based incident account.

This module supports LO2, LO3, LO6 and LO7.

1 Plan work around accepted outcomes

Agile development uses short feedback cycles to reduce the cost of discovering that a solution is wrong or unnecessary. A backlog item should describe a useful behaviour and its acceptance criteria. A sprint or short work cycle selects a bounded increment that can be reviewed and demonstrated.

For the energy service, an increment might be: when the advice provider fails, retain the verified energy report and mark advice unavailable. That is small enough to implement, test and explain. "Build the entire AI platform" is not a useful one-week lab item.

A definition of done states the evidence needed before the team considers an item complete. For this course it includes reviewed code, relevant passing tests, updated usage instructions, a recorded environment and no unresolved violation of the task contract. A demonstration alone is insufficient if another student cannot repeat it.

AI can draft tasks, identify dependencies or summarise a diff. A human still checks that the tasks represent the user's need and that completion claims match evidence. Do not equate the number of agent-generated commits with project progress. The unit of progress is accepted behaviour.

2 Collaboration and review capacity

Use small changes and clear ownership. A reviewer should know the requirement, affected interface and available checks. If several people work on the project, agree unit conventions, data schema and failure policies before independent implementation.

Work in progress is the number of unfinished items occupying the team. Starting many agent tasks can increase unfinished work and review queues. Limit concurrent changes to what the team can inspect and integrate. Waiting time, rework and conflicting changes can dominate any reduction in typing time.

A retrospective asks what happened, why and what one process change might improve the next cycle. Use observed examples: an ambiguous field caused two incompatible implementations; a smaller interface agreement could prevent recurrence. Avoid evaluating a teammate solely by visible code volume when review and debugging are essential contributions.

3 Continuous integration delivery and deployment

Continuous integration, or CI, means integrating changes frequently and checking the combined result automatically. Continuous delivery keeps a validated release ready for deployment, often with a separate release decision. Continuous deployment automatically releases changes that pass the established gates. These are related but distinct practices.

The core quality gate for this course prepares the environment, checks syntax, runs deterministic tests and records the outcome. A richer pipeline can add static analysis, dependency scanning, integration tests and a small AI evaluation smoke suite. Each check should address a stated risk.

```
Proposed change
-> source review
-> environment and dependency check
-> deterministic tests
-> integration and failure tests
-> selected AI regression cases
-> release decision and version record
```

A pipeline is code and needs review. If an AI task edits the tests or pipeline to skip a failure, inspect whether the change is justified by a revised requirement. Removing the evaluator can create a green status without improving the software.

4 A repeatable local quality gate

The supplied `run_checks.py` executes the standard-library tests and checks that the examples compile. Run it from the examples folder using the approved Python interpreter. Its nonzero exit status indicates failure, which a CI service can use to stop a release.

```
python run_checks.py
```

This local command is the required core path. To run it in a hosted CI service, check out the repository, select the documented Python version and execute the same command. Keep credentials out of untrusted pull-request jobs and give the workflow only the permissions it needs. Hosted configuration syntax and action versions should be checked against current official documentation before use. [R1]

Compilation checks catch syntax errors but not incorrect calculations. Unit tests do not prove a live provider's quality. A small AI smoke suite can catch obvious response-contract regressions but does not replace the project's final evaluation. Explain the coverage boundary of each gate.

5 Environment and dependency evidence

Record the operating environment, interpreter version and direct dependencies. A lock or resolved dependency record captures specific versions so another installation can reproduce the intended set. The record is useful only if installation actually respects it.

Pinning versions improves repeatability but can preserve a vulnerable release indefinitely. Combine a stable environment with a process for evaluating updates. Inspect dependency identity, source, necessity and maintenance. An AI-suggested package name may be incorrect or resemble another package; verify it before installation.

A software bill of materials, or SBOM, inventories software components. Build provenance records how an artifact was produced, from which source and process. An inventory does not prove that components are safe, and provenance does not prove that the source implements the correct requirement. They answer different traceability questions. SLSA provides a versioned framework for source and build assurances. [R2]

For the student project, retain a small dependency manifest and a release record with source commit, build or preparation command, environment and output checksum. Do not claim a formal SLSA level without evaluating all its requirements.

6 Deployment choices

Deployment places a selected version into an execution environment. For a local application, a clean folder with documented installation and run instructions can be a legitimate teaching deployment. A container packages an application with parts of its runtime environment, but it still depends on the host and configuration. It is not a guarantee of isolation from every threat.

A managed service can reduce infrastructure administration while introducing provider configuration, billing and data-location decisions. Kubernetes coordinates containers across infrastructure, but its operational complexity is unnecessary for the core project. Choose it only when an extension has a clear requirement and the baseline already works.

Separate configuration from code. Environment-specific paths, service endpoints and credentials should not require editing the calculation. Validate required configuration at startup. A missing AI credential should produce an explicit unavailable-advice state if the application supports a deterministic mode, not a misleading success.

Keep the release immutable enough to identify what was run. Editing files directly in a deployed folder without recording the change destroys that evidence. Promote a known artifact or source revision and preserve the ability to return to it.

7 Observability

Observability concerns the ability to understand internal behaviour from system outputs. Logs record events, metrics aggregate measurements, and traces follow a request across operations. OpenTelemetry documents these signal categories and their relationship. [R3]

For an energy report, a useful log includes a request identifier, release identifier, event name, processing outcome and duration. A metric might count invalid batches, advice fallbacks or accepted reports. A trace might connect input parsing, retrieval, model call and response validation so a slow request can be located.

Avoid logging raw prompts, full manuals, credentials or identifiable user data by default. A request identifier can connect events without exposing content. If content capture is needed for an approved evaluation, define access, retention and redaction explicitly.

Metrics need denominators. Ten failures are ambiguous without the number of requests and the time interval. Measure advice-fallback rate as fallback events divided by eligible advice requests, not all page views. Define whether timeouts and invalid responses count as failures in the service objective.

8 Service objectives and error budgets

A service-level indicator is a measured quantity such as the proportion of valid requests completed within a target duration. A service-level objective is the target for that indicator over a stated window. An error budget is the permitted amount of failure implied by the objective.

For an illustrative objective of 99% successful deterministic reports over 1000 valid requests, up to 10 failures fit the request-count budget. If the system has 12 failures, it misses that objective. Define eligible requests and success before counting; excluding failures after observing them would distort the metric.

A time-based availability objective uses a different denominator. For a 30-day period of 43,200 minutes, 99.9% availability permits 43.2 minutes of unavailability under that definition. Do not mix request-success and time-availability budgets as if they measure the same thing.

An exhausted error budget is evidence for reviewing reliability work and release risk. It is not an automatic legal or contractual rule. The course uses objectives as engineering decision tools.

9 Recovery and rollback

Rollback restores a previous application version or configuration. Restoring data is a separate operation. If a new release changes the database schema incompatibly, simply reverting the source may not restore operation. Plan migration and compatibility before release.

For the local prototype, keep a known-good release and a disposable copy of the synthetic dataset. Introduce a controlled configuration fault, observe the failure and restore the known version. Then rerun an acceptance scenario. A statement that rollback is possible is weaker evidence than a completed rehearsal.

A backup is useful only if restoration works. Record what was backed up, when, how it can be restored and how restored correctness is checked. Avoid testing recovery on the sole copy of valuable data.

10 Incidents as evidence for improvement

An incident account should state impact, timeline, detection, cause, recovery and follow-up actions. Distinguish known facts from hypotheses. "The AI made a mistake" is not a sufficient root cause if the application gave it unrestricted authority or failed to validate its response.

Example: a prompt update caused `source_ids` to be omitted. Response validation rejected advice, and the fallback kept numerical reports available. The incident affected advice availability, not energy correctness. The follow-up is a regression case for the missing field and a release check for prompt changes, rather than weakening validation to hide the failure.

NIST's SSDF supplies a general secure-development reference for preparation, protection, secure production and vulnerability response. [R4] Use the framework to identify responsibilities; this classroom project does not establish compliance with every recommended practice.

Weekly laboratories

Week 11 lab A quality gate and release record

- Minutes 0 to 10: write a definition of done for one project increment.
- Minutes 10 to 25: run the local quality gate and inspect the actual exit status.
- Minutes 25 to 40: introduce a harmless failing test in a disposable copy and verify that the gate stops.
- Minutes 40 to 50: restore the valid version and record interpreter, source and dependency information.
- Minutes 50 to 60: peer-review the release evidence and identify one check the gate does not perform.

Extension: run the same command in an approved hosted CI environment with minimal workflow permissions.

Week 12 lab Observe and recover

- Minutes 0 to 15: execute the end-to-end prototype and identify its release version.
- Minutes 15 to 30: inject a simulated provider failure and inspect the fallback event and metric.
- Minutes 30 to 45: restore the known configuration and rerun an acceptance case.
- Minutes 45 to 55: write a short incident account separating numerical availability from advice availability.
- Minutes 55 to 60: submit milestone 2 with the prototype, quality gate, evaluation plan and recovery evidence.

All injected failures use disposable local data. A container or paid cloud platform is an extension, not a substitute for the required evidence.

Practice questions

Question 1 Definition of done

Why should "the agent says complete" not be the team's definition of done?

Model answer. It is a claim rather than independently checked evidence. Completion should require the accepted behaviour, reviewed changes, relevant test results and reproducible instructions. The summary must agree with the actual release.

Question 2 Review queue

A team generates ten changes per day but reviews two. What problem is likely to grow?

Model answer. Unreviewed work accumulates, increasing integration delay, conflicts and forgotten context. Limit work in progress or improve review capacity. Generation rate alone does not measure delivery throughput.

Question 3 CI terminology

Distinguish continuous delivery from continuous deployment.

Model answer. Delivery keeps validated changes ready for release, often with a separate deployment decision. Deployment automatically releases changes that meet the established gates. Both depend on meaningful checks.

Question 4 Green pipeline

A pipeline passes after its integration test step is removed. What should a reviewer conclude?

Model answer. Only the remaining checks passed. Investigate why the step was removed and whether an equivalent check exists. The green status is not comparable to the earlier pipeline without considering coverage changes.

Question 5 Pinned dependencies

Does pinning every package version solve dependency security?

Model answer. No. It supports repeatability but may retain known vulnerabilities or an incorrect package. Verify identity and necessity, monitor updates and test planned upgrades.

Question 6 SBOM and provenance

What differs between a component inventory and build provenance?

Model answer. An inventory identifies included components. Provenance describes how an artifact was produced from source and a process. Neither alone proves secure behaviour or valid engineering requirements.

Question 7 Observability

Choose a log, metric and trace for an advice timeout.

Model answer. A log records the timeout event and request ID. A metric increments the timeout or fallback count with a defined denominator. A trace identifies the provider-call span within the complete request. Avoid unnecessary prompt or credential content.

Question 8 Request error budget

An objective requires 98% success over 500 valid requests. How many failures are allowed, and does 13 meet it?

Model answer. The allowed failure count is 2% of 500 = 10. Thirteen failures exceed the budget; success is $487/500 = 97.4\%$. The eligibility and success rules must have been defined beforehand.

Question 9 Time error budget

For a 24-hour window, calculate the downtime permitted by a 99.5% time-availability objective.

Model answer. There are 1440 minutes. The permitted unavailability is $0.005 \times 1440 = 7.2$ minutes. This is a time-based metric and cannot be substituted for a request success rate.

Question 10 Rollback

Why may reverting application code fail to recover from a release with a database migration?

Model answer. The old code may be incompatible with the changed schema or data. Recovery needs a compatible migration strategy, tested restore procedure or backward-compatible transition, followed by validation.

Question 11 Logging

An engineer proposes logging every prompt and API key to simplify debugging. Evaluate the proposal.

Model answer. API keys should not be logged. Prompt content may contain restricted information and requires a specific approved capture policy. Prefer request IDs, versions, durations and controlled error categories, with restricted redacted content only when necessary.

Question 12 Incident interpretation

Advice fails but verified energy reports remain available. What distinction belongs in the incident report?

Model answer. State that advice availability was affected while deterministic reporting continued, if the evidence supports that. Identify the fallback, duration, number of affected requests and cause. Avoid describing the entire system as either fully successful or fully unavailable without component detail.

Revision checklist

Explain what a quality gate checks and omits. Reproduce a release from its instructions. Distinguish inventory from provenance. Define a metric denominator. Calculate an error budget. Rehearse restoration rather than merely describing it. Write an incident account with observable facts and a targeted follow-up test.

Sources and further reading

- [R1] GitHub. GitHub Actions documentation. <https://docs.github.com/en/actions>
- [R2] SLSA. Specification Version 1.2. <https://slsa.dev/spec/v1.2/>
- [R3] OpenTelemetry. Signals. <https://opentelemetry.io/docs/concepts/signals/>
- [R4] NIST. Secure Software Development Framework Version 1.1. <https://doi.org/10.6028/NIST.SP.800-218>
- Further reading: Google. Site Reliability Engineering, service-level objectives. <https://sre.google/sre-book/service-level-objectives/>

References checked 15 September 2026. Service targets and incident scenarios are illustrative teaching examples, not guarantees for a live platform.